

Structuring Tailwind CSS v4 with Node.js, Express, and EJS: A Comprehensive Guide

xAI

June 26, 2025

Contents

1	Introduction	2
2	Overview of Components	2
3	Prerequisites	2
4	Project Structure	2
5	Step-by-Step Setup	2
5.1	Initialize the Project	2
5.2	Install Dependencies	3
5.3	Configure Tailwind CSS	3
5.4	Set Up Express and EJS	3
5.5	Update package.json	4
5.6	Run the Application	4
6	Best Practices	5
7	Troubleshooting	5
8	Conclusion	5

1 Introduction

This guide provides a detailed walkthrough for structuring a web application using Tailwind CSS v4, Node.js, Express, and EJS, without a Tailwind configuration file, as successfully implemented on June 26, 2025. Tailwind CSS offers utility-first styling, Node.js enables server-side JavaScript, Express simplifies web server creation, and EJS facilitates dynamic HTML rendering. This setup is ideal for building modern, styled web applications with minimal configuration.

2 Overview of Components

- **Tailwind CSS v4:** A utility-first CSS framework that uses a CLI to compile styles without a configuration file, leveraging automatic content detection for efficiency (Tailwind CSS).
- **Node.js:** A JavaScript runtime for server-side execution, known for its non-blocking I/O model (Node.js).
- **Express:** A minimalist web framework for Node.js, handling routing and middleware (Express).
- **EJS:** A templating engine that embeds JavaScript in HTML for dynamic server-side rendering (EJS).

3 Prerequisites

Before starting, ensure you have:

- Node.js (v18.x or later) installed.
- npm (included with Node.js).
- A code editor (e.g., Visual Studio Code).
- Basic understanding of JavaScript, HTML, and CSS.

4 Project Structure

A clear project structure enhances maintainability. The recommended layout is:

5 Step-by-Step Setup

5.1 Initialize the Project

Create a new directory and initialize a Node.js project:

```
1 mkdir tailwind-express-ejs
2 cd tailwind-express-ejs
3 npm init -y
```

Directory/File	Purpose
public/	Stores static assets like CSS and JavaScript.
public/css/style.css	Input file with Tailwind directives.
public/css/output.css	Compiled Tailwind CSS output.
views/	Contains EJS templates for server rendering.
views/index.ejs	Main EJS template for the home page.
index.js	Main server file, sets up Express.
package.json	Project metadata and dependencies.

Table 1: Project File Structure

5.2 Install Dependencies

Install the necessary packages:

```
1 npm install express ejs @tailwindcss/cli
```

This includes Express for the server, EJS for templating, and the Tailwind CSS CLI for styling.

5.3 Configure Tailwind CSS

- Create `public/css/style.css` with Tailwind directives:

```
1 @tailwind base;  
2 @tailwind components;  
3 @tailwind utilities;
```

- Run the Tailwind CLI to compile styles:

```
1 npx @tailwindcss/cli -i public/css/style.css -o public/css/  
  output.css --watch
```

This generates `public/css/output.css` and watches for changes in development.

5.4 Set Up Express and EJS

- Create `index.js` to set up the Express server:

```
1 import express from "express";  
2  
3 const app = express();  
4  
5 app.set("view engine", "ejs");  
6 app.set("views", "./views");  
7 app.use(express.json());  
8 app.use(express.static("public"));  
9 app.use(express.urlencoded({ extended: false }));  
10  
11 app.get("/", (req, res) => {  
12   res.render("index.ejs");  
13 });
```

```
14
15 app.listen(3000, () => {
16   console.log("Server is running on port 3000");
17 });
```

- Create `views/index.ejs` with:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-
      scale=1.0">
6   <title>Document</title>
7   <link rel="stylesheet" href="/css/output.css">
8 </head>
9 <body>
10   <h1 class="font-mono bg-black text-teal-500 p-4">HI IAm H D
      Rutvik</h1>
11 </body>
12 </html>
```

5.5 Update package.json

Modify `package.json` to include scripts:

```
1 {
2   "name": "tailwind-express-ejs",
3   "version": "1.0.0",
4   "type": "module",
5   "scripts": {
6     "start": "node index.js",
7     "build": "npx @tailwindcss/cli -i public/css/style.css -o
      public/css/output.css",
8     "watch": "npx @tailwindcss/cli -i public/css/style.css -o
      public/css/output.css --watch"
9   },
10  "dependencies": {
11    "express": "^5.1.0",
12    "ejs": "^3.1.10",
13    "@tailwindcss/cli": "^4.1.10"
14  }
15 }
```

5.6 Run the Application

- In one terminal, compile Tailwind CSS:

```
1 npm run watch
```

- In another terminal, start the Express server:

```
1 npm start
```

- Visit `http://localhost:3000` to see the styled page.

6 Best Practices

- Use `-watch` during development to auto-update CSS.
- Organize static assets in `public/` for easy access via Express.
- Test routes and templates separately to isolate issues.
- Use version control (e.g., Git) to track changes.
- Consider adding Nodemon (`npm install nodemon`) for auto-restarting the server:

```
1 npx nodemon index.js
```

7 Troubleshooting

- **Page loads indefinitely:** Check server logs for errors. Ensure `app.set("view engine", "ejs")` and `app.set("views", "./views")` are included.
- **Styles not applied:** Verify `public/css/output.css` is generated and linked correctly in `index.ejs`.
- **Port conflict:** Check if port 3000 is in use with `lsof -i :3000` and change to another port (e.g., 3001) if needed.
- **Syntax errors:** Ensure middleware like `express.json()` uses parentheses.

8 Conclusion

This guide outlines a successful setup for integrating Tailwind CSS v4 with Node.js, Express, and EJS, as achieved on June 26, 2025. The structure is scalable, allowing for additions like PostgreSQL or REST APIs. Use this as a reference for maintaining or replicating the project.